

Piscine React Native

TP aligne a la grille d'evaluation

Parcours guide

Parcours libre

Grille 100 pts

0 · Règles de rendu & organisation GitHub

GitHub & Branches

- Dépôt commun (1 par groupe) — branche **main** = version finale stable
- 1 branche par étudiant : *mobile-ui-<prenom>*, *api-<prenom>*, *native-<prenom>*...
- Commits clairs et réguliers (pas de *update*, *fix*...)

README.md requis

- Présentation courte (objectif, fonctionnalités clés)
- Installation & lancement (pré-requis, commandes)
- Architecture : schémas navigation, services, composants
- Modèles de données, vues/écrans, services/API, composants natifs
- ≥ **5 captures** : Caméra, Carte, Calendrier, Liste/Filtre, Profil
- Tableau répartition des tâches

Organisation & Livrables

- Kanban / Taiga / Trello ou GitHub Projects (Backlog → Doing → Review → Done)
- Livrables : dépôt + slides PDF + export issues/board (optionnel)

1 · Règles d'évaluation



La soutenance, c'est principalement le Q&A.;

La majorite de la note repose sur votre capacite a repondre aux questions. Une belle application ne sauvera personne : si vous ne comprenez pas votre code, vous n'avez pas de note. Pas d'excuse, pas de recours.

Ce sur quoi vous serez interrogés

Votre code	Chaque ligne que vous avez écrite. Pas le code de votre équipier — le vôtre. On vous demandera d'expliquer, modifier ou débbugger en direct.
Vos choix techniques	Pourquoi ce state manager ? Cette lib ? Cette architecture ? "C'est ce qu'on a trouvé sur internet" n'est pas une réponse.
Les composants natifs	Caméra, GPS, Carte, Calendrier — comment ça marche vraiment, pas juste comment vous l'avez copié-collé.

La gestion des données	Context, reducer, AsyncStorage, flux offline. Savoir tracer une donnée de l'UI jusqu'au cache.
Les bugs & limites	Ce qui ne marche pas et pourquoi. L'honnêteté technique est valorisée. Prétendre que tout est parfait quand ce n'est pas le cas, non.

—

Des points négatifs sont possibles.

Un manque d'effort visible — application vide, README absent, slides bâclées, code copié sans compréhension — peut entraîner un malus sur votre note globale. Il n'y a pas de plancher. Rendre quelque chose de honteux peut coûter plus cher que de ne rien rendre.

Ce qui est quand même pris en compte

Application technique	Qualité du code, navigation, API, composants natifs, gestion offline. Ce n'est pas la majorité de la note, mais un projet cassé ou vide sera sanctionné.
Documentation	README complet, schémas lisibles, répartition des tâches honnête. Si c'est bâclé, ça se voit — et ça compte.
Présentation	Structure, slides, timing, expression. Accessoire par rapport au Q&A, mais une soutenance chaotique aggrave une mauvaise impression.

2 · Parcours 1 — TP guidé

Journal de voyage interactif

Objectif

- Application qui prend des photos, géolocalise et date chaque prise, les affiche sur une carte.
- Offre un calendrier pour filtrer les jours et une liste filtrable par date/lieu.
- Inclut un écran profil avec statistiques.

■ Caméra

- Capturer, prévisualiser, sauvegarder métadonnées (uri, lat/lon, date)
- Stockage local AsyncStorage/SQLite + synchro optionnelle API mock

■ Calendrier

- Jours marqués si photos présentes
- Sélectionner une date = filtrer la liste

■ Profil & Stats

- Infos utilisateur
- Statistiques : nb photos, nb lieux, jour le plus actif...

■ API & Service

- photosService, locationService, statsService
- Mock API JSON Server + gestion erreurs (retry, état)

■ Localisation & Carte

- Récupérer position GPS
- Carte avec marqueurs ; appui = modal aperçu + lien détail

■ Liste / Galerie

- FlatList + filtres (date, rayon géo, mot-clé)
- Tri récent → ancien, pagination/infinite scroll

■ Navigation

- Tabs : Caméra | Carte | Calendrier | Photos | Profil
- Stack vers Détail photo + transmission de params
- Drawer pour Profil / déconnexion

■ Offline & Sécurité

- Cache AsyncStorage + file outbox à la reconnexion
- .env pour clés, validation basique des entrées

■ Critères d'acceptation (extraits)

- Prendre une photo crée un objet Photo avec **takenAt** + (**lat**, **lon**)
- La carte affiche 1 marqueur par photo ; appui = modal avec aperçu
- Le calendrier badge les jours avec photos ; tap = filtrage de la liste
- La FlatList affiche les photos filtrées ; filtres date/lieu fonctionnels
- README inclut ≥ 5 captures, schémas et tableau de répartition

■ Déroulé soutenance (15 min max)

1	Contexte & user stories
2	Démo guidée des écrans
3	Architecture (navigation, services, natifs, offline)
4	Choix techniques / limites / suites
5	Q&A;

3 · Parcours 2 — TP libre

Projet personnel contraint

Principe

- Choisissez votre thème (Gestion de reçus, Carnet d'entraînement, Journal culinaire...)
- Vous devez respecter les contraintes minimales ci-dessous pour couvrir tout le barème.

Contraintes minimales

Fonctionnel

- 3 composants natifs minimum (utilisés utilement)
- Navigation : Tabs + Stack, params, header personnalisé
- Entité principale : CRUD visible en UI
- FlatList + filtres + tri
- Écran de détail avec média + action
- Profil/Stats : ≥ 2 indicateurs agrégés

Technique

- Service/API : REST + gestion erreurs (loading/error/empty)
- Séparation : services/, state/, screens/, components/
- 1–2 tests d'utilis ou de services

Livrables

- GitHub + README complet + schémas
- ≥ 5 captures d'écran
- Slides PDF

Exemples de thèmes

■ Receipts Tracker

Photo reçu (caméra) + montant + lieu (GPS) + historique + carte dépenses + calendrier achats

■ City Explorer

Lieux favoris avec photo, carte, filtres tags/date, calendrier de visites, stats

■ Workout Journal

Séance avec photo/vidéo, calendrier des entraînements, carte des parcours, statistiques

■ Garden Diary

Photos de plantes, calendrier arrosage/entretien, carte du jardin

4 · Gabarits & Artefacts attendus

A. Gabarit README

1	Pitch & User Stories
2	Stack & pré-requis
3	Installation & run (dev, mock API, build)
4	Architecture (schémas inclus)
5	Modèle de données (entité principale + attributs)
6	Fonctionnalités livrées (checklist)
7	Sécurité & confidentialité (permissions, .env)
8	Répartition des tâches (tableau)
9	Captures d'écran (≥ 5)

B. Schémas requis (2 minimum)

- Navigation : écrans, stacks, tabs, flux
- Services & dataflow : UI → Service → API/Cache, offline, erreurs

C. Slides (≤ 10 slides)

- 1 — Contexte & user stories
- 2 — Démo live ou vidéo (gif/captures)
- 3 — Architecture
- 4 — Choix & arbitrages
- 5 — Limites & suites
- 6 — Répartition & bilan

■ **Échéance**

Vendredi 8h — Dépôt GitHub + Release v1.0 + Slides PDF

Bon courage — soignez la démonstration autant que le code ! ■